

CP Equation Simplification Cheat Sheet

Competitive Programming identities, transformations, and recognition patterns

How to use this file: Do not memorize every formula blindly. When you see a large expression, ask: Can I count per bit, per element, per pair, per edge, or by prefix? Then search for the matching identity below.

Core example

$$R(x, y) = \text{popcount}(x \mid y) - \text{popcount}(x \& y) = \text{popcount}(x \wedge y)$$

Why: At each bit, the difference is 1 only when exactly one of x and y has that bit set. That is precisely XOR.

Contents

1. Bitwise and Popcount Identities
2. XOR over Ranges and Sequences
3. Floor, Ceil, Division, and Modulo
4. Arithmetic Sums and Progressions
5. Counting Pairs, Intervals, and Contributions
6. GCD, LCM, and Divisibility
7. Modular Arithmetic Patterns
8. Combinatorics Identities
9. Probability and Expected Value
10. Graph and Tree Equations
11. String Length and Matching Identities
12. DP and Optimization Transformations
13. Common Algebraic Rewrites
14. Fast Recognition Checklist

1. Bitwise and Popcount Identities

Identity / Simplification	Why it is true	Typical CP use
$\text{popcount}(x \mid y) - \text{popcount}(x \& y) = \text{popcount}(x \wedge y)$	Bits set in exactly one of x and y.	Hamming distance, differing features, mismatched masks.
$\text{popcount}(x) + \text{popcount}(y) = \text{popcount}(x \mid y) + \text{popcount}(x \& y)$	Each bit is counted once or twice on both sides.	Replace a sum of popcounts by union/intersection counts.
$x + y = (x \wedge y) + 2 \cdot (x \oplus y)$	XOR is addition without carries; AND locates carries.	Bitwise equations, carry analysis, constructive problems.
$x \wedge y = x + y - 2 \cdot (x \oplus y)$	Rearrangement of the carry identity.	Convert XOR into arithmetic when AND is constrained.
$x \mid y = (x \wedge y) + (x \oplus y)$	XOR and AND contain disjoint set bits.	Turn OR into a sum when overflow is irrelevant.
$x \mid y = x + y - (x \& y)$	Equivalent arithmetic form.	Simplify objectives containing OR and AND.
$x \& y = x + y - (x \mid y)$	Rearrangement of the OR identity.	Recover intersection from sum and union.
$(x \mid y) \wedge (x \& y) = x \wedge y$	OR and AND never share an uncertain bit.	Useful in XOR transforms and proof simplifications.
$x \wedge x = 0, \quad x \wedge 0 = x$	XOR cancels equal values.	Prefix XOR, parity, finding an unpaired value.
$x \wedge y \wedge y = x$	Applying the same XOR twice restores x.	Encoding/decoding, reversible transitions.
$x \& x = x, \quad x \mid x = x$	Idempotence.	Remove repeated masks.
$x \& 0 = 0, \quad x \mid 0 = x$	Neutral/absorbing elements.	Boundary simplification.
$x \& (x - 1)$ removes the lowest set bit	All lower trailing zeros/one transition are cleared.	Iterating set bits, checking powers of two.
$x \& (-x)$ isolates the lowest set bit	Two's complement leaves only the least significant 1.	Fenwick tree, lowbit, binary decomposition.
$x > 0$ and $(x \& (x - 1)) = 0 \Leftrightarrow x$ is a power of two	Only one set bit exists.	Power-of-two tests.
$x \mid (x - 1)$ sets all bits below the lowest set bit	The suffix becomes all ones.	Mask construction and next-state tricks.
$x \wedge (x - 1) = 2 \cdot \text{lowbit}(x) - 1$	The changed suffix is exactly a block of ones.	Reasoning about binary increments.
submask enumeration: $s = (s - 1) \& \text{mask}$	Moves to the next smaller submask.	Enumerate all submasks in $O(3^n)$ over all masks.
number of submasks of mask = $2^{\{\text{popcount}(\text{mask})\}}$	Every set bit is independently chosen.	Subset counting and SOS DP.
number of ordered pairs (a,b) with $a \mid b = \text{mask} = 3^{\{\text{popcount}(\text{mask})\}}$	Each set bit goes to a only, b only, or both.	Bitmask combinatorics.
number of ordered pairs (a,b) with $a \& b = 0$ inside n bits = 3^n	Per bit: neither, a only, or b only.	Disjoint-mask counting.
$a \subseteq b$ as masks $\Leftrightarrow (a \& b) = a$	Every bit of a also appears in b.	Subset relation on bitmasks.
a and b are disjoint $\Leftrightarrow (a \& b) = 0$	No common set bit.	Compatibility tests.

Identity / Simplification	Why it is true	Typical CP use
$\text{parity}(\text{popcount}(x)) = x_0 \wedge x_1 \wedge \dots$	XOR of all bits equals popcount modulo 2.	Parity constraints and linear algebra over GF(2).

2. XOR over Ranges and Sequences

Identity / Simplification	Why it is true	Typical CP use
$a_l \wedge \dots \wedge a_r = \text{prefXor}[r] \wedge \text{prefXor}[l-1]$	Equal prefix elements cancel.	O(1) range XOR queries.
$0 \wedge 1 \wedge \dots \wedge n$ follows $n \bmod 4$: [n, 1, n+1, 0]	The pattern repeats every four values.	XOR of integer ranges.
$\text{xor}(l..r) = \text{xor}(0..r) \wedge \text{xor}(0..l-1)$	Prefix cancellation.	Range XOR without an array.
XOR of every element appearing an even number of times = 0	Pairs cancel.	Frequency-parity arguments.
XOR of all subset XORs is 0 when $n \geq 2$	Each element appears in 2^{n-1} subsets, an even count.	Global subset-XOR simplifications.
Sum of XOR over all unordered pairs = $\sum_{\text{bit}} \text{cnt1} \cdot \text{cnt0} \cdot 2^{\text{bit}}$	A pair contributes at a bit iff its bits differ.	Compute pairwise XOR sum in $O(n \log V)$.
Sum of AND over all unordered pairs = $\sum_{\text{bit}} C(\text{cnt1}, 2) \cdot 2^{\text{bit}}$	Both values must contain the bit.	Pairwise AND sum.
Sum of OR over all unordered pairs = $\sum_{\text{bit}} [C(n, 2) - C(\text{cnt0}, 2)] \cdot 2^{\text{bit}}$	A pair contributes unless both miss the bit.	Pairwise OR sum.
For fixed x : $\sum_i (x \wedge a_i) = \sum_{\text{bit}} \text{mismatches}_{\text{bit}} \cdot 2^{\text{bit}}$	Each bit contributes independently.	Choose x bit-by-bit to minimize/maximize XOR cost.
Minimum $\sum_i (x \wedge a_i)$: choose each bit by majority	At each bit, pay the smaller of ones and zeros.	Bitwise median under Hamming-weighted distance.
Maximum $\sum_i (x \wedge a_i)$: choose each bit by minority complement	At each bit, force the larger mismatch count.	Maximization under independent bits.

3. Floor, Ceil, Division, and Modulo

Identity / Simplification	Why it is true	Typical CP use
$\text{ceil}(a / b) = (a + b - 1) / b$ for integers $a \geq 0, b > 0$	Integer-division formula.	Blocks, pages, batches, segment counts.
$\text{ceil}(a / b) = a / b + (a \% b \neq 0)$ for $a \geq 0$	Adds one only when a remainder exists.	Avoid overflow from $a+b-1$.
$\text{floor}(a / b)$ counts multiples of b in $[1, a]$	Exactly $b, 2b, \dots, \lfloor a/b \rfloor b$.	Counting divisibility.
multiples of k in $[l, r] = \lfloor r/k \rfloor - \lfloor (l-1)/k \rfloor$	Prefix-count subtraction.	Range divisibility queries.
numbers congruent to $c \bmod k$ in $[l, r] = \lfloor (r-c)/k \rfloor - \lfloor ((l-1)-c)/k \rfloor$	Shift the residue class to multiples of k .	Count arithmetic progressions in a range.
$a = q \cdot b + r$, where $q =$	Euclidean division for $b > 0$.	Proofs involving quotient and remainder.

Identity / Simplification	Why it is true	Typical CP use
$\text{floor}(a/b), 0 \leq r < b$		
$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$	Reduce operands early.	Prevent overflow and normalize expressions.
$(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$	The +m prevents a negative representative.	Safe modular subtraction.
$(a \cdot b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$	Multiplication respects congruence.	Modular products.
$a \equiv b \pmod{m} \Leftrightarrow m \text{ divides } (a-b)$	Definition of congruence.	Replace modular equality by divisibility.
$a \bmod m = a - m \cdot \text{floor}(a/m)$	Remainder expressed using floor.	Convert between floor sums and modulo sums.
$\text{floor}((a+b)/m) = \text{floor}(a/m) + \text{floor}(b/m) + \text{carry}$	carry is 1 iff $(a \bmod m) + (b \bmod m) \geq m$.	Carry counting and cyclic shifts.
$\text{floor}(n/k)$ changes only $O(\sqrt{n})$ times as k varies	For small k use direct values; for large k quotients are small.	Floor-division grouping.
For fixed l , $q = \text{floor}(n/l)$, maximal r with same q is $\text{floor}(n/q)$	All indices $l..r$ share one quotient.	$O(\sqrt{n})$ harmonic loops.
$\sum_{i=1}^n \text{floor}(n/i) = \sum_{d=1}^n \tau(d)$	Both count pairs (i,j) with $i \cdot j \leq n$.	Divisor-sum transformations.
$\sum_{i=1}^n (n \bmod i) = n^2 - \sum_{i=1}^n i \cdot \text{floor}(n/i)$	Use $n \bmod i = n - i \lfloor n/i \rfloor$.	Modulo-sum problems.

4. Arithmetic Sums and Progressions

Identity / Simplification	Why it is true	Typical CP use
$1 + 2 + \dots + n = n(n+1)/2$	Sum of first n integers.	Counting pairs, triangular numbers.
$1 + 3 + \dots + (2n-1) = n^2$	Sum of first n odd numbers.	Square-growth arguments.
$2 + 4 + \dots + 2n = n(n+1)$	Twice the triangular sum.	Even-index sums.
$1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6$	Square-sum formula.	Distance/cost sums and variance.
$1^3 + 2^3 + \dots + n^3 = [n(n+1)/2]^2$	Cube sum is the square of triangular sum.	Polynomial summation.
Arithmetic progression sum = $\text{count} \cdot (\text{first} + \text{last})/2$	Pair terms from both ends.	Range sums with constant difference.
$k + (k+d) + \dots + (k+(n-1)d) = nk + d \cdot n(n-1)/2$	Split constant and index parts.	Weighted linear sequences.
Geometric sum $1+r+\dots+r^{n-1} = (r^n-1)/(r-1), r \neq 1$	Multiply by r and subtract.	Repeated multiplication, doubling, matrix ideas.
Infinite geometric sum = $a/(1-r), r < 1$	Limit of the finite formula.	Expected values and probabilities.
$\sum_{i=l}^r r^i = T(r) - T(l-1)$	Prefix sum of integers.	$O(1)$ interval sums.
$\sum_{i=l}^r i^2 = S_2(r) - S_2(l-1)$	Prefix square-sum subtraction.	$O(1)$ quadratic interval costs.
$\sum_{i=0}^{n-1} (a+id) = na + d \cdot n(n-1)/2$	General arithmetic progression.	Index-based simplification.

Identity / Simplification	Why it is true	Typical CP use
$\sum_i (a_i - c)^2 = \sum a_i^2 - 2c\sum a_i + n c^2$	Expand the square.	Fast squared-distance evaluation.
$\sum_i a_i - x $ is minimized by a median	Moving x toward the median decreases more distances than it increases.	L1 optimization.
$\sum_i (a_i - x)^2$ is minimized by the mean	Derivative/complete-square argument.	L2 optimization.
For sorted a : $\sum a_i - x $ can be split by lower_bound	Left side contributes $x \cdot \text{cnt} - \text{sum}$; right contributes $\text{sum} - x \cdot \text{cnt}$.	$O(\log n)$ absolute-distance queries.

5. Counting Pairs, Intervals, and Contributions

Identity / Simplification	Why it is true	Typical CP use
number of unordered pairs from n items = $C(n, 2) = n(n-1)/2$	Choose two distinct items.	Pair counting.
number of ordered distinct pairs = $n(n-1)$	Choose first then second.	Directed interactions.
number of subarrays of length n = $n(n+1)/2$	Choose left and right endpoints.	All-subarray counts.
number of subarrays containing index i (0-based) = $(i+1)(n-i)$	Choose left endpoint $\leq i$ and right endpoint $\geq i$.	Contribution technique.
number of subarrays where $[l, r]$ is contained = $(l+1)(n-r)$	Extend left and right independently.	Interval contribution.
number of subarrays of length exactly k = $n-k+1$	Possible starting positions.	Fixed-window counts.
number of subarrays of length at least k = $(n-k+1)(n-k+2)/2$	Sum counts for lengths $k..n$.	Threshold-length counting.
In a sorted array, pair contribution to $\sum_{i < j} (a_j - a_i)$: a_i coefficient = $2i - n + 1$	Appears positive i times and negative $n-1-i$ times.	$O(n)$ sum of pairwise absolute differences.
$\sum_{i < j} a_i - a_j $ for sorted a = $\sum_i a_i (2i - n + 1)$	Sorted order removes absolute values.	Pairwise distance sum.
For frequency f , equal unordered pairs = $C(f, 2)$	Choose any two occurrences.	Duplicate-pair counting.
Pairs with different values = $C(n, 2) - \sum_v C(\text{freq}_v, 2)$	Complement of equal pairs.	Diversity statistics.
Number of inversions = $\sum$ over elements of previous greater elements	Count each inversion by its right endpoint.	Fenwick/merge-sort interpretation.
$\sum$ over all subarrays of a_i = $\sum_i a_i (i+1)(n-i)$	Element contribution times number of containing subarrays.	All-subarray sum in $O(n)$.
For maxima/minima contributions: count choices until previous/next blocking element	Each element owns subarrays where it is the selected extreme.	Monotonic-stack sums.
Total edge contribution in all tree paths = $\text{size} \cdot (n - \text{size})$	Removing the edge splits endpoints into two components.	Sum of all pairwise tree distances.

6. GCD, LCM, and Divisibility

Identity / Simplification	Why it is true	Typical CP use
$\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$	Euclidean algorithm.	Fast GCD and algebraic reductions.
$\text{gcd}(a, b) = \text{gcd}(a, b-a)$ when $b \geq a$	Common divisors are unchanged by subtraction.	Proofs and transformations.
$\text{gcd}(a, b) \cdot \text{lcm}(a, b) = ab $	Prime exponents use $\min + \max = \text{sum}$.	Convert between GCD and LCM.
$\text{gcd}(a, b) = g \rightarrow a = gx, b = gy, \text{gcd}(x, y) = 1$	Factor out the common gcd.	Normalize Diophantine constraints.
$\text{gcd}(a_1, \dots, a_n)$ divides every integer linear combination	Any $\sum c_i a_i$ is a multiple of the gcd.	Reachability by additions/subtractions.
$ax + by = c$ has integer solutions iff $\text{gcd}(a, b)$ divides c	Bézout's identity.	Linear Diophantine equations.
If $\text{gcd}(a, m) = 1$, then a has a modular inverse mod m	Coprimality is exactly the invertibility condition.	Modular division.
For prime p and a not divisible by p : $a^{p-1} \equiv 1 \pmod{p}$	Fermat's little theorem.	Inverse a^{p-2} , exponent reduction.
Euler: $\text{gcd}(a, m) = 1 \Rightarrow a^{\phi(m)} \equiv 1 \pmod{m}$	Generalized exponent cycle.	Composite moduli.
Number of divisors of $n = \prod p_i^{e_i}$ is $\prod (e_i + 1)$	Choose each prime exponent independently.	Divisor counting.
Sum of divisors of $n = \prod p_i^{e_i}$ is $\prod (1 + p_i + \dots + p_i^{e_i})$	Independent geometric choices.	Divisor-sum functions.
Number of multiples of d up to n is $\text{floor}(n/d)$	Direct counting.	Inclusion-exclusion and divisor loops.
$\sum_{d n} \phi(d) = n$	Classify numbers $1..n$ by gcd with n .	Totient transformations.
$\text{gcd}(a^m - 1, a^n - 1) = a^{\text{gcd}(m, n)} - 1$	Exponent Euclidean structure.	Number theory exponent problems.
If $a b$ and $b c$, then $a c$	Divisibility is transitive.	Constraint chains.

7. Modular Arithmetic Patterns

Identity / Simplification	Why it is true	Typical CP use
$a^b \bmod m$: reduce b modulo $\phi(m)$ only when $\text{gcd}(a, m) = 1$	Euler reduction needs coprimality.	Avoid invalid exponent simplification.
For prime p : $C(n, k) \bmod p$ may use factorials only while $n < p$	Factorials become zero modulo p at p .	Know when Lucas theorem is needed.
Inverse of a mod prime $p = a^{p-2} \bmod p$	From Fermat's theorem.	Division under prime modulus.
$(a/b) \bmod m$ means $a \cdot b^{-1} \bmod m$, not integer division	Modular division is multiplication by an inverse.	Combinatorics and probabilities modulo m .
Normalize $x \bmod m$ as $((x \% m) + m) \% m$	Produces a representative in $[0, m-1]$.	Languages with negative remainder.
$a^0 = 1$, including modulo m unless $m = 1$ where result is 0	Identity exponent with modulus edge case.	Fast power correctness.

Identity / Simplification	Why it is true	Typical CP use
If $a \equiv b \pmod{m}$, then $f(a) \equiv f(b)$ for integer-coefficient polynomials f	Congruence is preserved by + and $\times$.	Replace values by residues.
Chinese remainder for coprime moduli has a unique solution modulo product	Residues combine independently.	Merge cyclic constraints.
$x^2 \equiv 1 \pmod{\text{prime } p} \Rightarrow x \equiv \pm 1 \pmod{p}$	$(x-1)(x+1)$ divisible by p .	Quadratic congruence simplification.
Modulo 2, addition and subtraction are both XOR	Because $1+1 \equiv 0$.	Parity DP and GF(2).

8. Combinatorics Identities

Identity / Simplification	Why it is true	Typical CP use
$C(n, k) = C(n, n-k)$	Choose selected items or excluded items.	Reduce k and exploit symmetry.
$C(n, k) = C(n-1, k) + C(n-1, k-1)$	Split by whether a fixed item is chosen.	Pascal recurrence and DP.
$k \cdot C(n, k) = n \cdot C(n-1, k-1)$	Mark one chosen element.	Simplify weighted binomial sums.
$(n-k) \cdot C(n, k) = n \cdot C(n-1, k)$	Mark one unchosen element.	Weighted binomial identities.
$\sum_{k=0}^n C(n, k) = 2^n$	Count all subsets.	Subset sums.
$\sum_{k=0}^n k \cdot C(n, k) = n \cdot 2^{n-1}$	Each element appears in half of all subsets.	Total subset size.
$\sum_{k=0}^n k^2 \cdot C(n, k) = n(n+1) \cdot 2^{n-2}$	Use $k^2 = k(k-1) + k$ and mark elements.	Second moments of subset size.
$\sum_k C(r, k) C(s, n-k) = C(r+s, n)$	Choose n objects from two groups.	Vandermonde convolution.
$\sum_{i=k}^n C(i, k) = C(n+1, k+1)$	Hockey-stick identity.	Prefix sums of combinations.
Number of nonnegative solutions to $x_1 + \dots + x_k = n$ is $C(n+k-1, k-1)$	Stars and bars.	Distribute identical objects.
Positive solutions to $x_1 + \dots + x_k = n$: $C(n-1, k-1)$	Give one to each variable first.	Compositions.
Permutations of multiset counts c_i : $n! / \prod c_i!$	Divide by swaps of equal items.	Anagrams and duplicate arrangements.
Catalan $C_n = C(2n, n) / (n+1)$	Counts balanced structures.	Parentheses, noncrossing matchings, BST shapes.
Derangements $D_n = (n-1)(D_{n-1} + D_{n-2})$	Track where item 1 goes and whether a 2-cycle forms.	Permutations with no fixed points.
Inclusion-exclusion: $ U A_i = \sum A_i - \sum A_i \cap A_j + \dots$	Corrects overcounting by alternating signs.	Forbidden-property counting.
Burnside: number of orbits = average number of fixed configurations	Count objects modulo symmetry.	Rotations/reflections and necklaces.

9. Probability and Expected Value

Identity / Simplification	Why it is true	Typical CP use
$E[X+Y] = E[X]+E[Y]$	Linearity holds without independence.	Break a hard count into indicator variables.
For indicator $I[\text{event}]$, $E[I] = P(\text{event})$	I is 1 exactly when the event occurs.	Expected counts.
Expected number of successful objects = $\sum$ probabilities of success	Apply indicators and linearity.	Random subsets, collisions, coverage.
$P(A \cup B) = P(A) + P(B) - P(A \cap B)$	Inclusion-exclusion for two events.	Union probability.
$P(A B) = P(A \cap B) / P(B)$	Definition of conditional probability.	Update after information is known.
Bayes: $P(A B) = P(B A)P(A) / P(B)$	Reverse a condition.	Inference problems.
If independent: $P(A \cap B) = P(A)P(B)$	One event does not change the other.	Product probabilities.
$E[X] = \sum_{k \geq 1} P(X \geq k)$ for nonnegative integer X	Count how many thresholds X passes.	Stopping times and maximum-like variables.
$\text{Var}(X) = E[X^2] - E[X]^2$	Expand $E[(X - E[X])^2]$.	Variance computation.
For independent X, Y : $\text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y)$	Covariance vanishes.	Sums of independent trials.
Geometric waiting time with success probability p has expectation $1/p$	One-step recurrence $E = 1 + (1-p)E$.	Expected retries.
Coupon collector expectation = $n(1 + 1/2 + \dots + 1/n)$	When k collected, next new coupon probability is $(n-k)/n$.	Collect-all processes.

10. Graph and Tree Equations

Identity / Simplification	Why it is true	Typical CP use
$\sum_v \text{degree}(v) = 2m$	Every undirected edge contributes two endpoints.	Handshake lemma.
Number of odd-degree vertices is even	The degree sum is even.	Parity proofs.
Tree with n vertices has $n-1$ edges	Connected and acyclic characterization.	Tree validation and counting.
In a forest: components = $n-m$	Each added forest edge reduces components by one.	DSU and cycle detection.
$\text{distance}(u, v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}(u, v)]$	Remove the shared root-to-LCA path twice.	Tree distance queries.
Path aggregate with invertible prefix: $\text{pref}[u] + \text{pref}[v] - 2\text{pref}[\text{lca}] + \text{value}[\text{lca}]$	Combine root paths and correct the LCA.	Path sums on trees.
For weighted tree, total pairwise distance = $\sum_{\text{edges}} w \cdot \text{size} \cdot (n - \text{size})$	Each crossing pair uses the edge.	All-pairs distance sum.
Cyclomatic number of connected graph = $m - n + 1$	Extra edges beyond a spanning tree create independent cycles.	Cycle-space dimension.
For DAG, number of paths can be	Every path chooses its first outgoing edge.	Path counting.

Identity / Simplification	Why it is true	Typical CP use
$DP[v] = \sum DP[to]$ in reverse topological order		
Bipartite graph: $ maximum\ matching = minimum\ vertex\ cover $	Kőnig's theorem.	Convert matching and covering objectives.
Flow conservation: $inflow(v) = outflow(v)$ except source/sink	Internal vertices neither create nor destroy flow.	Network-flow modeling.
Max-flow = min-cut	The tightest cut limits all flows and is attainable.	Prove optimality of flow solutions.

11. String Length and Matching Identities

Identity / Simplification	Why it is true	Typical CP use
Number of substrings of a length- n string = $n(n+1)/2$	Choose start and end.	Substring counting.
Number of subsequences = 2^n (including empty)	Choose or skip each position.	Subset interpretation of subsequences.
Number of nonempty subsequences = $2^n - 1$	Exclude the empty choice.	Basic counting.
$LCP(i, j) \geq k \iff s[i..i+k-1] = s[j..j+k-1]$	Definition of longest common prefix.	Binary search with hashing / suffix structures.
For rolling hash: $H(l, r) = pref[r] - pref[l] \cdot base^{r-l}$	Remove the prefix before l after alignment.	$O(1)$ substring hash.
Palindrome test by hash: $forwardHash(l, r) = reverseHash(n-r, n-l)$	Compare the interval with its mirrored interval.	Fast palindrome queries.
Borders of s are lengths k with $prefix(k) = suffix(k)$	A border is both prefix and suffix.	KMP prefix-function applications.
Occurrences of pattern p in text t can be found in $p + \#t$	Prefix/Z values crossing $ p $ mark matches.	KMP/Z black-box recognition.
Minimum characters appended to make palindrome = $n - \text{longest palindromic suffix}$	Keep the longest suffix already mirrored.	Palindrome completion.
Distinct substrings = $n(n+1)/2 - \sum LCP \text{ of adjacent suffixes}$	Each suffix adds length minus overlap with previous suffix.	Suffix-array counting.
Suffix automaton distinct substrings = $\sum_v (len[v] - len[link[v]])$	Each state represents a new interval of substring lengths.	SAM counting.
Manacher total palindromic substrings = sum of radii contributions	Each radius level represents one palindrome.	Count all palindromic occurrences in $O(n)$.

12. DP and Optimization Transformations

Identity / Simplification	Why it is true	Typical CP use
$dp[i] = \min_j (dp[j] + cost(j, i))$	Generic transition: choose the previous cut/state.	Recognize optimization structure.
If $cost(j, i) = A_j \cdot B_i + C_j$, transition is min of lines at $x = B_i$	Separate j-only slope/intercept and i-only query.	Convex Hull Trick / Li Chao.
$(x-y)^2 = x^2 - 2xy + y^2$	Separates terms by source and destination.	Turn quadratic DP into line queries.
$dp[i] = \min_j (dp[j] + (a_i - a_j)^2) \Rightarrow$ query line slope $-2a_j$	Expand the square.	Classic CHT transformation.
Prefix-sum segment cost: $sum(l..r) = pref[r] - pref[l-1]$	Remove repeated range summation.	DP partition costs.
Knapsack choice count: each item contributes choose/skip	State transition follows inclusion/exclusion of item.	0/1 knapsack reasoning.
For bounded item count c , split c into powers of two	Any number $0..c$ is representable by binary groups.	Convert bounded knapsack to $O(\log c)$ 0/1 items.
SOS DP: $F[mask] = \sum_{sub \subseteq mask} A[sub]$	Aggregate over submasks dimension by dimension.	Subset-query preprocessing.
Subset convolution intuition: split mask into disjoint A and $mask \setminus A$	Each partition assigns every bit to one side.	DP over set partitions.
Reroot relation often has form $ans[child] = ans[parent] + lost - gained$	Only contributions crossing the moved root change.	Derive rerooting formulas.
Binary search on answer requires monotone predicate: $false \dots false, true \dots true$	Feasibility persists after a threshold.	Turn optimization into decision.
Meet-in-the-middle reduces 2^n to about $2 \cdot 2^{n/2}$	Split choices into two halves.	Subset problems around $n \approx 40$.

13. Common Algebraic Rewrites

Identity / Simplification	Why it is true	Typical CP use
$a(b+c) = ab+ac$	Distributive law.	Factor or expand to expose structure.
$ab+ac = a(b+c)$	Factor the common term.	Reduce repeated computation.
$a^2 - b^2 = (a-b)(a+b)$	Difference of squares.	Divisibility and factorization.
$(a+b)^2 = a^2 + 2ab + b^2$	Square expansion.	Distance/cost transformations.
$(a-b)^2 = a^2 - 2ab + b^2$	Square expansion.	CHT and least-squares formulas.
$a^3 - b^3 = (a-b)(a^2 + ab + b^2)$	Difference of cubes.	Factor divisibility expressions.
$a^3 + b^3 = (a+b)(a^2 - ab + b^2)$	Sum of cubes.	Factor polynomial constraints.
$1+2+\dots+(n-1) = C(n, 2)$	Triangular sum equals pair count.	Switch between algebra and combinatorics.
$\min(a, b) = (a+b - a-b)/2$	Subtract the difference from the sum.	Algebraic min expression.
$\max(a, b) = (a+b + a-b)/2$	Add the difference to the sum.	Algebraic max expression.

Identity / Simplification	Why it is true	Typical CP use
$\max(a, b) + \min(a, b) = a + b$	The two values are merely reordered.	Simplify symmetric expressions.
$\max(a, b) - \min(a, b) = a - b $	Distance between the two values.	Range and spread formulas.
$ x = \max(x, -x)$	Definition via two linear pieces.	Piecewise optimization.
$ a - b = a + b - 2 \cdot \min(a, b)$	Assume one is smaller and simplify.	Transform absolute differences.
$ a - b = 2 \cdot \max(a, b) - a - b$	Equivalent max form.	Contribution techniques.
$\sum_i \sum_j f(i, j)$ can often swap order: $\sum_j \sum_i f(i, j)$	Finite sums are reorderable.	Count by the easier dimension.
Count objects = $\sum$ indicators	Each object contributes 1 if valid, else 0.	Expected value and contribution proofs.
Optimization over independent bits/coordinates separates into per-bit/per-coordinate choices	No cross-term couples decisions.	XOR costs, Hamming distance, Manhattan distance.

14. Fast Recognition Checklist

Identity / Simplification	Why it is true	Typical CP use
Expression contains $x+y$ and $x \& y$	Try $x+y = (x \wedge y) + 2(x \& y)$.	Bitwise carry decomposition.
Expression contains $\text{popcount}(\text{OR})$, $\text{popcount}(\text{AND})$, $\text{popcount}(\text{XOR})$	Use $\text{popcount}(x) + \text{popcount}(y) = \text{popcount}(\text{OR}) + \text{popcount}(\text{AND})$ and $\text{OR} - \text{AND} = \text{XOR}$ in popcount form.	Collapse multiple popcounts.
Sum over all pairs	Try contribution by element, bit, edge, or sorted position.	Avoid $O(n^2)$.
Sum over all subarrays	Count how many subarrays contain each index or make it min/max.	Contribution/monotonic stack.
Many $\text{floor}(n/i)$ terms	Group equal quotients with $r = n/(n/l)$.	$O(\sqrt{n})$ loops.
Range count of multiples/residues	Use prefix floor counts.	$O(1)$ formula.
Quadratic DP transition	Expand the square and search for a line form.	CHT/Li Chao.
Tree all-pairs sum	Cut each edge and count endpoint pairs across it.	Edge contribution.
Random expected count	Create one indicator per object/event.	Linearity of expectation.
Subset counting with bit conditions	Treat each bit independently; common bases are 2 or 3 choices per bit.	$2^k / 3^k$ patterns.
Absolute-distance objective	Sort; median for minimization; prefix sums for queries.	L1 geometry.
Squared-distance objective	Expand; mean minimizes; maintain $\sum a$ and $\sum a^2$.	L2 geometry.
Modular division	Replace division with multiplication by inverse and verify invertibility.	Correct modular algebra.
GCD constraint	Factor out g and make remaining variables coprime.	Normalize number-theory equations.

Identity / Simplification	Why it is true	Typical CP use
Repeated substring equality	Use hashing/LCP/suffix structures rather than character comparison.	String-query reduction.

Important Conditions and Traps

- **Integer ceil formula:** $(a+b-1)/b$ is safe only for nonnegative a and positive b ; it may overflow, so $a/b + (a\%b!=0)$ is often safer.
- **Negative modulo:** C++ remainder can be negative. Normalize with $((x\%m)+m)\%m$ when you require $[0, m-1]$.
- **Exponent reduction:** Reducing an exponent modulo $\phi(m)$ generally requires $\gcd(\text{base}, m)=1$, unless you handle the non-coprime case separately.
- **Modular division:** You may divide by b modulo m only when b has an inverse modulo m , i.e. $\gcd(b, m)=1$.
- **Overflow:** Even if the final result fits in 64 bits, intermediate products such as $n(n+1)$ may overflow. Divide first when possible or use `__int128`.
- **Bit shifts:** Use `1LL<<k` for 64-bit masks. Shifting into or beyond the type width is invalid.
- **Pair formulas:** Check whether pairs are ordered/unordered and whether $i=j$ is allowed.
- **Indexing:** Contribution formulas differ between 0-based and 1-based indexing. This file states the indexing when it matters.